

反向传播学习笔记

公式推导 · 手算复盘 · 结构化整理版

学习来源：B 站视频《反向传播：海量参数的神经网络如何训练》（UP 主：谦行 Aling）

视频地址：<https://www.bilibili.com/video/BV1Lg5T6SEmP/>

原始笔记：2026 年 8 月 23 日 16:27 | 本整理版：2026 年 8 月 28 日

整理说明：

原笔记是边看视频边记的语音转文字（一行一句）。本次整理做了五件事：

- ① 把逐句碎片合并成连贯的段落，每一段对应它后面的那张截图；
- ② 修正转写错字（“reload”应为 ReLU、“偏执”应为偏置等，完整对照见附录 A）；
- ③ 颜色约定：灰色字 = 视频里的原话（人说的话），黑色字 = 提炼出来的重点与公式；
- ④ 新增前向传播汇总表、梯度链条表、参数更新表，方便复习；
- ⑤ 开篇收录我的 4 页手稿（自己手写的 CNN 结构与逐层梯度推导，含 softmax + 交叉熵）。

一、我的手稿：公式草纸推导

以下 4 页手稿是我自己手写的推导草稿：把视频里讲的反向传播，落到这次作业「语音情感识别 CNN」上——先设计网络结构（第 1 页），再从损失函数出发逐层推导梯度（第 2~4 页）。手稿在前、视频笔记在后，两边对照着看：手稿是落地，视频是原理。

手稿第 1 页 | ① 频谱图 + LeNet 风格 CNN。 1.1 结构：Convolution → ReLU Activation ×3 →

MaxPool → Flatten → Hidden → 2×Fully connected。1.2 网络结构与维度流：输入 [N, 1, 64, 128]

（后改为 [N, 16, 64, 128]）→ Conv2d(1→16, 3×3, pad=1)+ReLU → MaxPool2d(2) →

Conv2d(16→32)+ReLU → Conv2d(32→64)+ReLU → MaxPool2d(2) → Flatten →

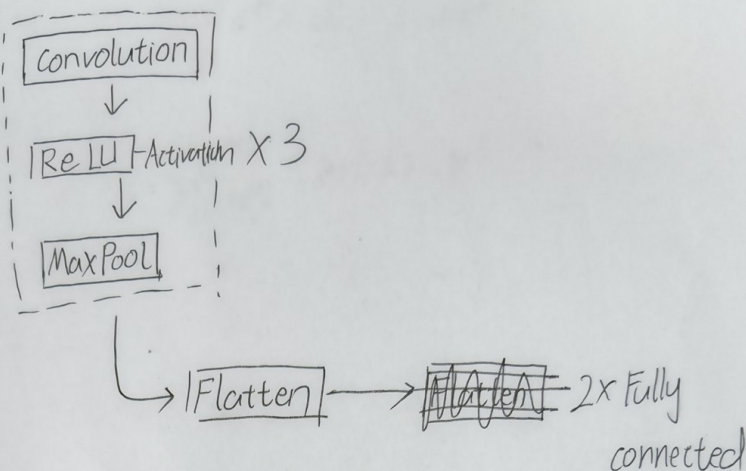
Linear(8192→128)+ReLU → Linear(128→8)，输出 8 类情感。维度列上的大 × 是自己标的错误记号：

第一版维度没对上——Linear 的输入 8192 要靠「通道数 × 池化后的空间尺寸」乘出来，重推时按这个口径对齐。

① 频谱图 + LeNet 风格 CNN

语音识别情感 speech emotion recognition.

1.1 结构:



1.2 网络结构与维度流

#	层	输出维度
0	输入	$[N, 1, 64, 128]$
1	Conv2d (1→16, 3x3, Pad=1) + ReLU	$[N, 16, 64, 128]$
2	MaxPool2d(2)	$[N, 16, 32, 64]$
3	Conv2d (16→32, 3x3, Pad=1) + ReLU	$[N, 32, 32, 64]$
4	Conv2d (32→64, 3x3, Pad=1) + ReLU	$[N, 64, 16, 32]$
5	MaxPool2d(2) (32→64, 3x3, Pad=1) + ReLU	$[N, 64, 8, 16]$
6	MaxPool2d(2)	
7	Flatten	
8	Linear (8192→128) + ReLU	
9	Linear (128→8)	

手稿 1 | ① 频谱图 + LeNet 风格 CNN: 结构与维度流

手稿第 2 页 | ② 目标函数——softmax 交叉熵。2.1 记号: N 个样本、 $K=8$ 类、logits $z^{(n)} \in \mathbb{R}^K$;

2.2 softmax 归一化 $P_k = e^{z_k} / \sum_j e^{z_j}$; 2.3 交叉熵 $L = (1/N) \sum_n \sum_k (-y_{nk} \cdot \ln P_{nk})$ 。3.1

开始推「输出层: softmax + 交叉熵的联合梯度」: 因为 P_j 依赖所有 z_k , 所以要用链式法则经 P 中转—— $\partial L / \partial z_k = \sum_j (\partial L / \partial P_j) \cdot (\partial P_j / \partial z_k)$, 并先写下两个偏导各自的结果。页底粉字「不懂」圈

的是 δ_{jk} : 它是 Kronecker 符号, $j=k$ 时取 1、否则取 0, 把求和按这两种情况拆开, 就能继续化简下去。

② 目标函数——softmax 交叉熵

2.1 $N, K=8, n, \text{ logits } \mathbf{z}^{(n)} = (z_1, \dots, z_K) \in \mathbb{R}^K$

2.2 softmax 归一化

$$P_k = \frac{e^{z_k}}{\sum_{j=1}^K e^{z_j}}, k=1, 2, 3, \dots, K$$

2.3 交叉熵

$$L = \frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K (-y_{nk} \ln P_{nk})$$

$$L = \frac{1}{N} \sum_{n=1}^N \sum_{k=1}^K (-y_{nk} \ln P_{nk})$$

③ 导数求解过程

3.1 输出层: softmax + 交叉熵的联合梯度

$$\text{结论 } \frac{\partial L}{\partial z_k} = P_k - y_k$$

推导: 单样本损失 $L = -\sum_j y_j \ln P_j$, 而 P_j 依赖所有 z_k , 所以要通
过 P_j 中较一般式求导

$$\frac{\partial L}{\partial z_k} = \sum_{j=1}^K \frac{\partial L}{\partial P_j} \cdot \frac{\partial P_j}{\partial z_k} \dots \text{① 链式法则}$$

$$\frac{\partial L}{\partial P_j} = \frac{-y_j}{P_j}; \quad \frac{\partial P_j}{\partial z_k} = P_j (\delta_{jk} - P_k) \dots \text{② 两个偏导各自的}$$

2 结果

手稿 2 | ② 目标函数: softmax + 交叉熵, 联合梯度推导

手稿第 3 页 | 3.1 推导收尾 + 3.2 全连接层。3.1 收尾: $\partial L / \partial z_k = \sum_j (-y_j)(\delta_{jk} - P_k) = -y_k + P_k \cdot \sum_j y_j = P_k - y_k$ (最后一步用了 one-hot 标签的 $\sum_j y_j = 1$) , 结论加了下划线; 旁边的红色? 是自己留的待确认标记。3.2 全连接层: $\mathbf{z} = \mathbf{W}\mathbf{a} + \mathbf{b}$, 链式法则给出三条公式—— $\partial L / \partial \mathbf{w}_{ij} =$

$\delta_i \cdot a_j$ 、 $\partial L / \partial b_i = \delta_i$ 、 $\partial L / \partial a_j = \sum_i \delta_i \cdot w_{ij}$ (δ_i 即误差项 = 损失对本层线性输出的导数) ; 配了一张 3 输入 $\rightarrow$ 4 神经元的结构图, 把 12 条连线——对应到公式下标; 最后归纳成通项 $a_j^{(l)} = \sigma(\sum_i w_{ij}^{(l)} \cdot a_i^{(l-1)} + b_j^{(l)})$ 和矩阵形式 $A^{(l)} = \sigma(W^{(l)} \cdot A^{(l-1)} + b^{(l)})$ 。

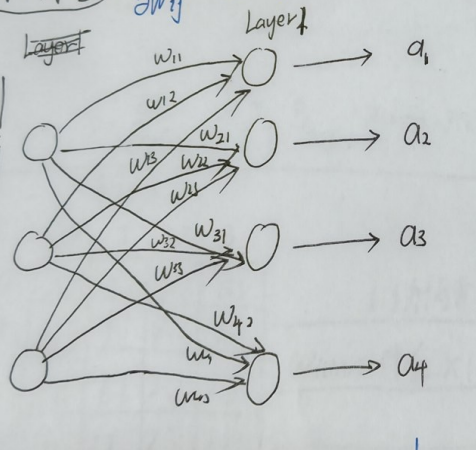
$$\frac{\partial L}{\partial z_k} = \sum_{j=1}^K \frac{\partial L}{\partial y_j} \cdot P_j (\delta_{jk} - P_k) = \sum_{j=1}^K (-y_j) (\delta_{jk} - P_k) = -y_k + P_k \sum_{j=1}^K y_j = P_k - y_k$$

$$\frac{\partial L}{\partial z_k} = \sum_{j=1}^K \frac{-y_j}{P_j} \cdot P_j (\delta_{jk} - P_k)$$

$$= \sum_{j=1}^K (-y_j) (\delta_{jk} - P_k) = -y_k + P_k \sum_{j=1}^K y_j \quad \left(\sum_{j=1}^K y_j = 1 \right)$$

$$= P_k - y_k$$

3.2 全连接层 $(z = Wa + b)$
 链式法则: $\frac{\partial L}{\partial w_{ij}} = \delta_i \cdot a_j$; $\frac{\partial L}{\partial b_i} = \delta_i$; $\frac{\partial L}{\partial a_j} = \sum_i \delta_i w_{ij}$



$a_1 = \sigma(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1)$
 $a_2 = \sigma(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2)$
 $a_3 = \sigma(w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3)$
 $a_4 = \sigma(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4)$

第 2 层第 i 个神经元的输出:

$$a_i^{(2)} = \sigma \left(\sum_{j=1}^n w_{ij}^{(2)} a_j^{(1)} + b_i^{(2)} \right)$$

$$A^{(2)} = \sigma(W^{(2)} A^{(1)} + b^{(2)})$$

手稿 3 | 3.1 结果 $P_k - y_k$; 3.2 全连接层三条梯度公式

手稿第 4 页 | 3.3 ReLU、3.4 池化、3.5 卷积层。3.3 ReLU: $z > 0$ 时导数为 1, 其余为 0。3.4

MaxPool: 只有前向时取到最大值的那个位置拿到梯度, 窗口内其余位置梯度为 0。3.5 Conv2d (单

通道、stride=1、无 padding) : 前向 $z_{ij} = \sum_{u,v} w_{uv} \cdot x_{i+u,j+v} + b$; 参数梯度 $\partial L / \partial w_{uv} = \sum_{i,j} \delta_{ij} \cdot x_{i+u,j+v}$; 输入梯度 $\delta_x = \text{conv_full}(\delta, W)$; 旁边画了 4×4 输入配 3×3 核、以 (i,j) 为锚点的对应关系。页底粉字的问题——「为什么要算对输入的梯度???'」——答案在下面。

3.3 ReLU $\sigma'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$ 简单 $\rightarrow$ ReLU

3.4 Pool Max

$\frac{\partial L}{\partial x_{u,v}} = \sum_{(i,j) \text{ 含 } (u,v) \text{ 的窗口 } (r,c) \text{ 的最大值, 其余梯度为 } 0}$

3.5 Conv2d 单通道, stride=1, 无 padding, 输入 x , 核 W 是 3×3

$$z_{ij} = \sum_{u,v} w_{uv} x_{i+u,j+v} + b, a_{ij} = \sigma(z_{ij})$$

$$\frac{\partial L}{\partial w_{uv}} = \sum_{i,j} \delta_{ij} \cdot x_{i+u,j+v}; \quad \frac{\partial L}{\partial b} = \sum_{i,j} \delta_{ij}$$

输入 x

	0	1	2	3
0	0.0	0.1	0.2	0.5
1	1.0	1.1	1.2	1.3
2	2.0	2.1	2.2	2.3
3	3.0	3.1	3.2	3.3

i,j 为锚点, $w_{u,v}$ 对位 $x_{i+u,j+v}$

Kernel

$w_{0,0}$	$w_{0,1}$	$w_{0,2}$
1.0	1.1	1.2
2.0	2.1	2.2

$\delta_{m,n} = \sum_{u,v} \delta_{i,j} w_{u,v}$, 其中 $i = m-u, j = n-v \Leftrightarrow \delta_x = \text{conv_full} \delta$

为什么要算对输入的梯度???

手稿 4 | 3.3~3.5: ReLU、MaxPool、Conv2d 的梯度

重点：为什么要算「对输入的梯度」？ 因为当前层的输入，就是上一层的输出。 $\partial L / \partial a_j^{(l-1)}$ 传回去，上一层参数 $w^{(l-1)}$ 的梯度才有的算（前层参数的梯度 = $\delta \times$ 本层输入）；不算它，链条在中间就断了，

前面所有层「断粮」、无法更新。只有真正的第一层可以不算——它前面已经没有参数了。这正是「七、链式法则」一节里「每条路径连乘、各路径相加」在这份手稿里的实际应用。

手稿与视频笔记的呼应：手稿第 2~3 页推出的 $\partial L / \partial z_k = P_k - y_k$ ，与视频笔记「八、手推反向传播」里 MSE 损失的 $\partial L / \partial y = y - t$ 处在同一个位置——都是「损失对输出层线性输出的导数」，形式同样极简（预测 - 真实）：一个走分类（softmax + 交叉熵），一个走回归（MSE）。

二、为什么要学反向传播

我们来看一下大名鼎鼎的反向传播。反向传播涉及到了一些数学运算，学习它会提升我们对于包括“激活函数的选择”“梯度下降”、各种各样优化器的作用，还有“梯度消失”“梯度爆炸”等问题的理解。这就是反向传播在神经网络中的核心作用，因为它诠释了一个神经网络是如何学习的。

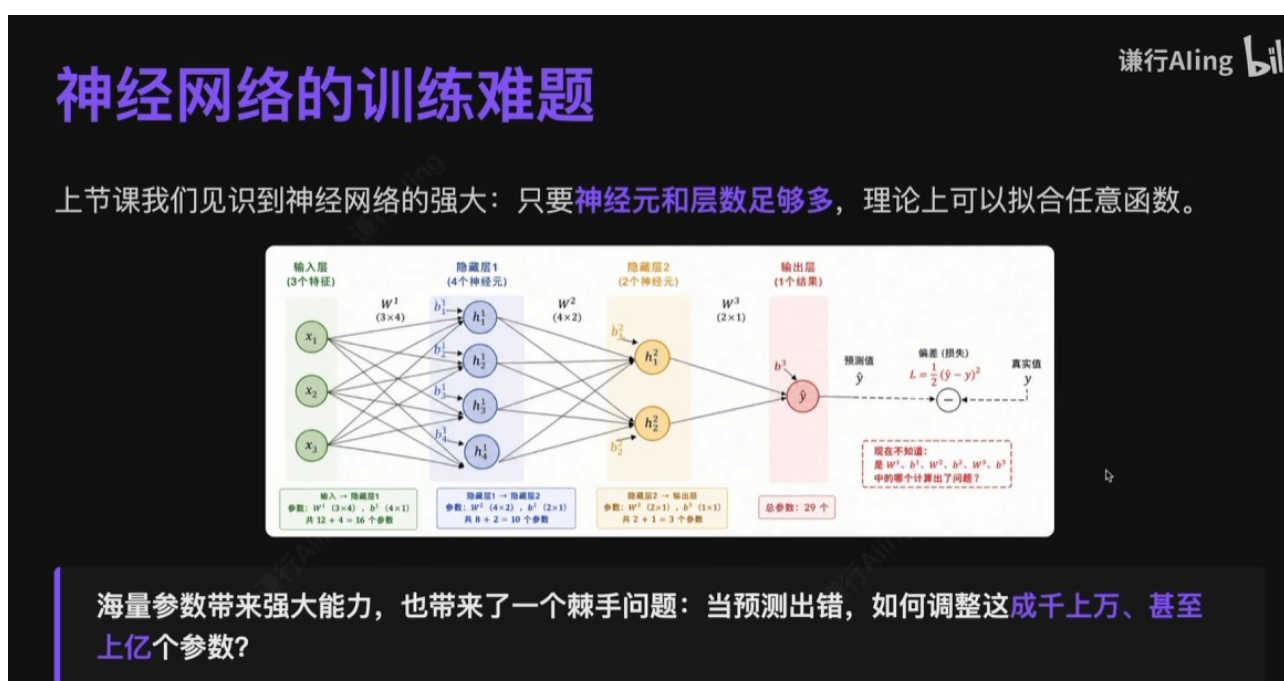


图 1 | 视频开场：反向传播的核心地位

重点：反向传播回答的核心问题是——预测出错之后，海量的参数到底该怎么调。

神经网络只要层数足够多，理论上来说，一个神经网络就可以拟合任意复杂的函数，做各种各样的预测。但是因为各个层之间的神经元是全连接的，每一条线就是一个权重的参数，最后每个神经元上还会有一个偏置的参数，即使这样一个简单的神经网络，参数也会达到 29 个。

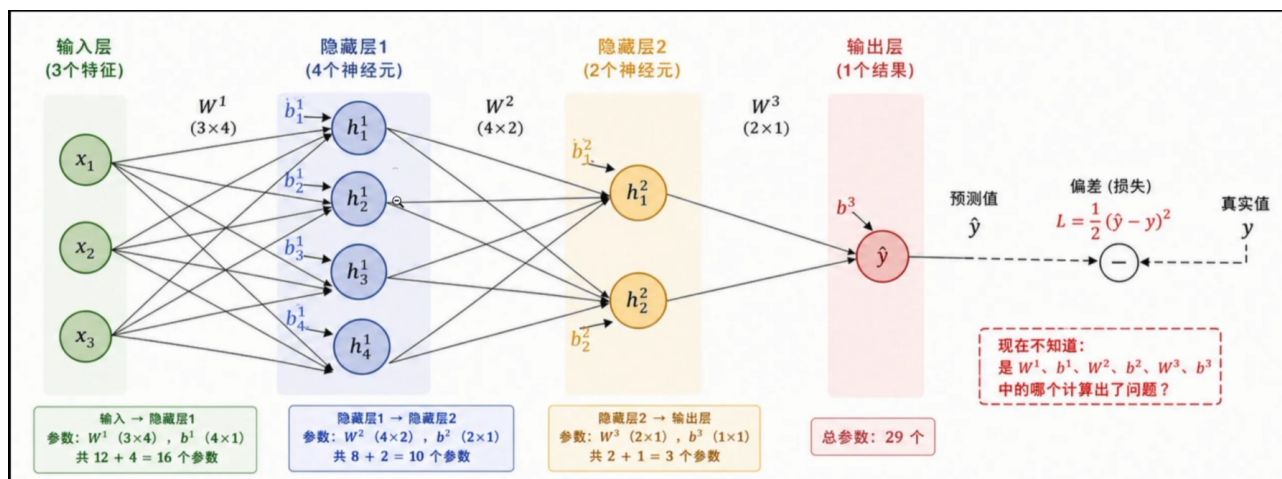


图 2 | 即使这样一个简单网络，参数也已多达 29 个

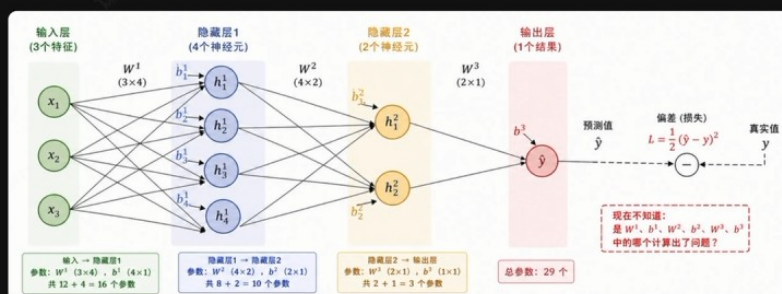
重点: 全连接结构：每条连线 = 一个权重，每个神经元再加一个偏置 → **参数量爆炸**。

我们实际用的神经网络，想让它模拟一些复杂的问题，可能需要成千上万甚至上亿个参数。当模型训练时发现预测值和真实值有差距，就需要调整参数了。那我们怎么调整这上亿个参数？到底应该调整谁、不应该调整谁？这就会是一个非常大的问题了。而正是这个问题，也导致神经网络进入了 20 年的寒冬——网络越深，参数越多，没法为每个参数手工求导了。

神经网络的训练难题

谦行Aling bilibili

上节课我们见识到神经网络的强大：只要**神经元和层数足够多**，理论上可以拟合任意函数。



海量参数带来强大能力，也带来了一个棘手问题：当预测出错，如何调整这**成千上万、甚至上亿个参数**？

这就会是一个非常大的问题了

图 3 | 参数上亿、无法手工求导：神经网络的 20 年寒冬

重点: 参数太多没法手工求导 → 神经网络陷入约 20 年的寒冬。

直到 1986 年，辛顿 (Hinton) 等人提出把反向传播算法跟深度学习做了结合：利用链式法则把误差从输出层逐层返回，就可以算出所有的权重，这才让神经网络变得可以训练。我们今天就来理解一下神经网络当发现预测出错的时候，该怎么调整参数的问题，这就是反向传播最大的作用。

反向传播的诞生

谦行Aling bilibili

多层网络的训练难题，因一个算法而破解。

困境

网络越深参数越多，无法为每个参数手动求导

突破

1986 年，Hinton、Rumelhart、Williams 提出反向传播算法

核心思想

利用链式法则，误差从输出层逐层传回，一次性算出所有梯度

图 4 | 1986 年，Hinton 等人让反向传播与深度学习结合

重点：1986 年 Hinton 等人：链式法则 + 误差逐层回传 → 一次性算出所有权重的梯度，神经网络从此“可训练”。

三、直观理解：公司“层层追责”的类比

在啃数学公式之前，我们先来看一个类比。假如一个公司发布了一个产品，发布之后口碑崩盘，产生了巨大的损失，CEO 震怒，他怎么办呢？他不会随机开除一个员工，也不会把所有员工一个个地问责，效率都太低了。公司特别庞大，CEO 一般是怎么办的？他是层层追责：先找直接向我汇报的下属，你们每个人来看一下到底是怎么回事；各个部门都领到了一定的责任，每个部门再往回去看它的子部门，看哪些部门应该承担对应的多大的责任。这样层层地追溯，直到追溯到最后，我们才会发现，有些人确实做错了一些事情，有些人并没有做错。对于做错的，我们就做一些调整；对于没有做错的，可以继续保持。

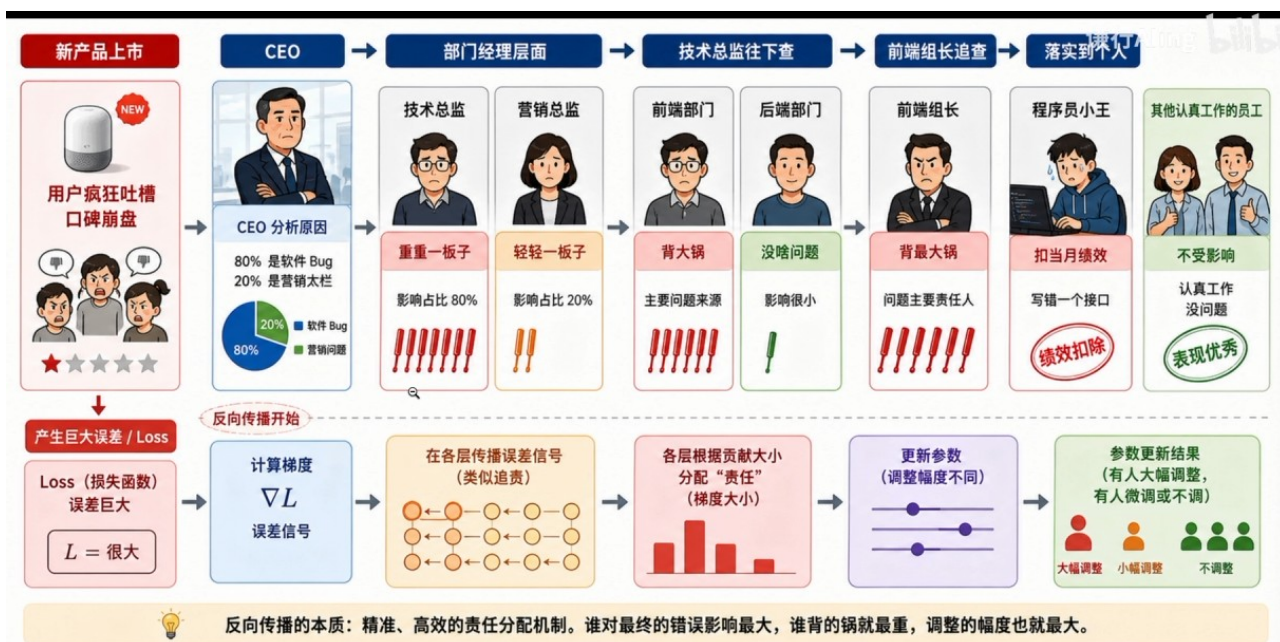


图 5 | 类比：产品口碑崩盘后，CEO 层层追责

这个就可以类比反向传播：当神经网络预测出现了巨大的误差之后，从最后的结果开始得出误差是多少，在各个层之间倒着往回追责，根据每层对这个错误的贡献度——谁的责任其实就是导数——根据责任不同，我们更新参数的幅度也是不同的。对于大责任的参数，就把它调得大一些；小责任就调得小一点，这样渐渐地就能够让预测更准一点。所以说反向传播其实就是一个层层追责：从最终的结果误差出发，倒着逐层看责任分配，看每一层、每一个神经元应该对这个误差负多大的责任；谁对误差影响越大，参数调整就越大。

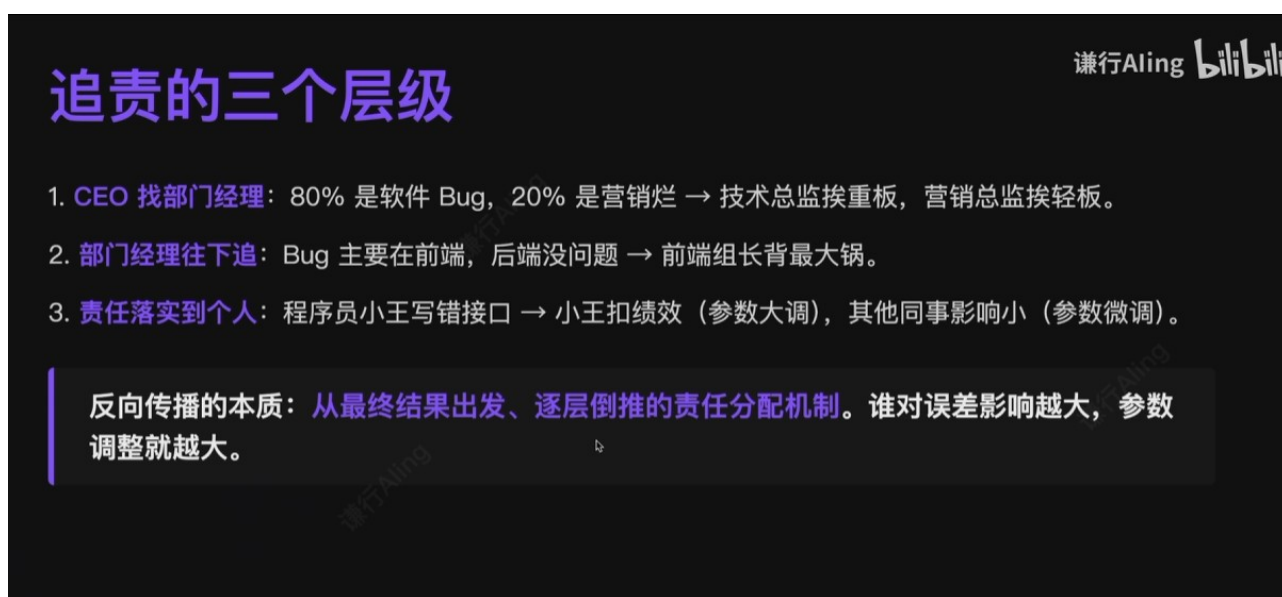


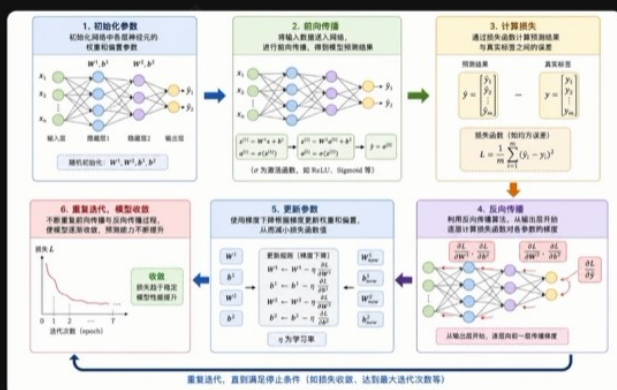
图 6 | 反向传播的本质：为每个参数定“责任”

重点：反向传播 = 为每个参数“定责任”的算法：从最终误差出发、倒着逐层分配责任；对误差影响越大 → 责任越大 → 调整幅度越大。定责任（算梯度）和执行处罚（更新参数）是分开的，后者交给梯度下降。

四、神经网络训练的完整流程

想彻底理解反向传播，首先要理解一个神经元训练过程中的几个步骤。

神经网络训练全流程



1. 初始化各层神经元的**权重和偏置**参数；
2. 输入数据做**前向传播**，得到预测结果；
3. 用**损失函数**计算预测与真实标签的误差；
4. 用**反向传播**从输出层逐层计算损失对参数的梯度；
5. 用**梯度下降**根据梯度更新权重和偏置；
6. 不断重复，让模型逐步收敛。

图 7 | 神经元训练过程的几个步骤

首先第一步，我们需要初始化参数。一个神经元默认参数都是零，这样是没法训练的，可以用一些初始化的工具给各个参数做初始化工作。有了初始化，下一步叫做前向传播：现在所有的参数都有了，我也有训练数据，虽然参数是随机的，就拿着训练数据让神经网络做一轮预测，从输入层开始层层算下去，直到算出预测值。接着对比预测的结果和真实标签的差距到底有多大。可能不同的模型有不同的计算损失的方法：对于回归问题，最常用的是均方差（MSE）；对于分类问题，可能用一些交叉熵——用不同的函数来计算损失有多大，这个函数就称之为损失函数，它算出来的值，就是这次预测这么多样本平均大概错了多少。再下一步才是反向传播——所以反向传播并不是模型训练最开始做的事情，它是在一个中间的位置：算出损失来之后，再开始反向传播，逐层计算每个神经元中的每个权重要为这个损失负多大的责任，也就是不断地算梯度：每一层、每个神经元、每个参数，损失函数对每个参数都要求导，一层层地把导数算出来，算出来之后就知道该负多大的责任了。然后就利用在机器学习里也讲过的梯度下降算法——可能会用各种各样的优化器——来更新参数。所以说反向传播也不是用来更新参数的，它只是告诉梯度下降，它们各负多大的责任、大概给每个参数分配多大的比例；具体怎么更新、更新多少，是梯度下降来决定的。更新了一轮参数之后，再进行迭代：用新的参数再进行前向传播，计算新的参数更新之后损失有多大，再反向传播、再更新，不断做这个循环。渐渐地我们会发现损失函数不断下降，损失不太变了、或者已经接近为零了、或者模型已经训练到极限了，就可以停下来，模型大概就是这个样子了。用最新的参数就可以正常预测，如果效果足够好，就可以在线上部署了。这就是一个神经网络训练的完整过程。

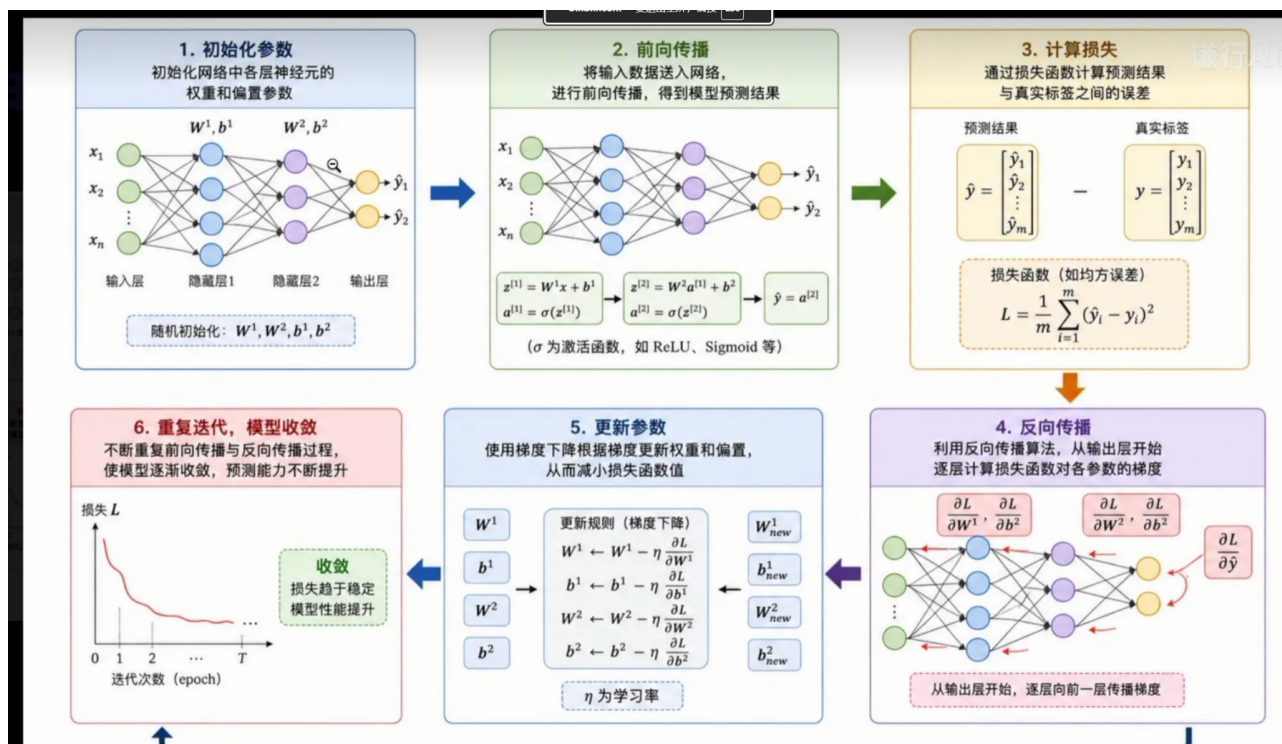


图 8 | 神经网络训练的完整循环

重点: 训练是一个五步循环——

1. **初始化参数** (不能全为 0, 用初始化工具赋随机初值) ;
2. **前向传播** (用训练数据做一轮预测) ;
3. **计算损失** (回归用 MSE, 分类用交叉熵) ;
4. **反向传播** (逐层算损失对每个参数的导数, 即责任) ;
5. **梯度下降更新参数**, 然后回到第 2 步循环, 直到损失足够小或基本不再变化。

重点: 两个易混点: ① 反向传播不是训练的第一步, 它在“算出损失之后”的中间位置; ② 反向传播只算梯度、不更新参数——具体更新多少由梯度下降 (优化器) 决定。

五、前向传播: 模型如何做预测

我们先来看一下前向传播。反向传播中有大量的公式, 这些公式用的参数、矩阵的形状都是怎么来的呢? 实际上是根据前向传播来的。前向传播简单而言, 可以理解为模型利用参数做预测的过程: 从输入层 (比如说这是一张图片) 提取出几个输入特征, 每个隐藏层都从自己的角度对上一层的神经元做一些组合和抽象——先做线性的变换, 加上一个非线性的激活, 再输出。每一层都从自己的角度对整个图片做一些理解, 对于上一层的神经元做一些组合和抽象, 这些组合和抽象就是一些数学公式, 形成了新的特征, 直到最后帮我们预测出结果——比如这是一只小猫咪。大家对机器学习里的 PCA 特

征提取、主成分分析、降维有印象的话，能很好地类比理解：把一些特征经过变换组成新的特征。正是这种逐层抽象的机制，让神经网络有了自动学习特征的能力。这也是深度学习和机器学习最大的区别：深度学习可以自己提取特征，不用人工做特征。

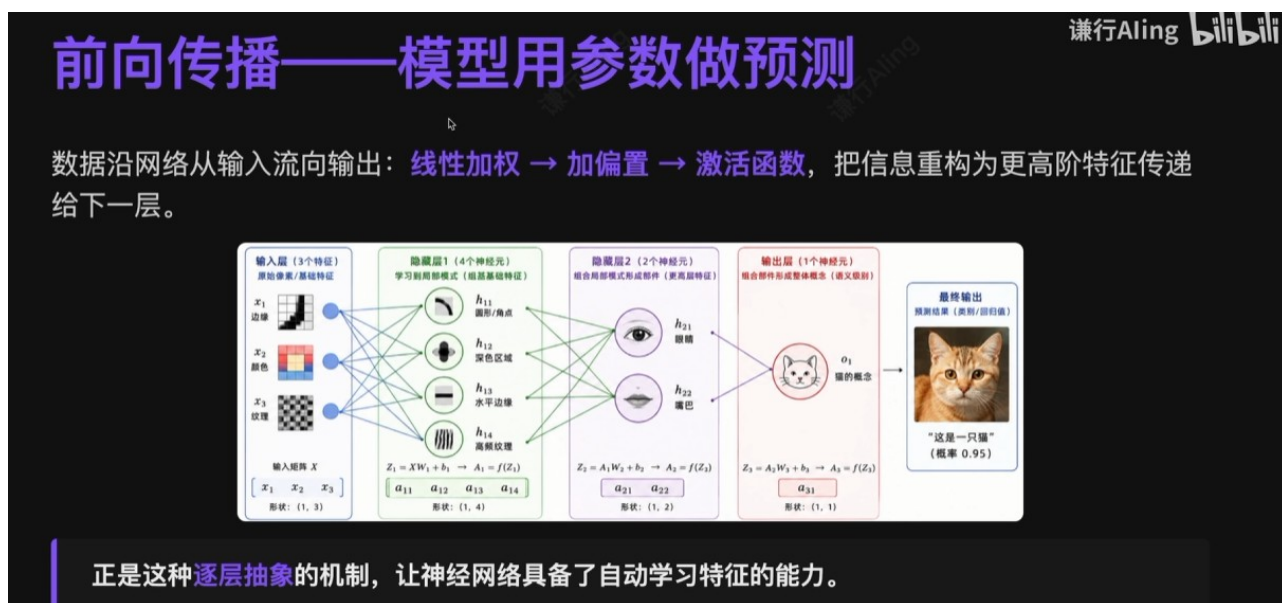


图 9 | 前向传播：逐层组合与抽象特征

重点：前向传播 = 模型用参数做预测；逐层抽象 = **自动特征提取**（深度学习与机器学习的最大区别）。

那我们来看一下过程中用的一些简单的数学。虽然神经元非常多，但每个神经元都是做同一套操作，分为两个步骤。比如神经元 a_1 ：上一层的三个输入都会连上 a_1 ，它对这三个输入都维护自己的权重参数，对它们做线性变换，自己再维护一个偏置——大家一看就知道，这就是一个线性回归的公式。做完线性变换之后，因为后面还需要进行各种叠加、各种层，如果只是线性变换，做再多叠加实际上都是没用的（上个视频也提到过），所以再做一次非线性激活。上一节课讲 MLP 的时候用的是阶跃函数；无论用什么激活函数，都可以用 σ 这个符号来代替它，把线性变换算出来的结果用非线性函数处理。如果非线性函数算出来有输出，就称它被激活了，否则就是被抑制了，所以这个函数也被称为激活函数。基本上就是这两个步骤，区别只是每个神经元都维护着自己的权重参数。比如 w_{11} ，前面的 1 代表它是这一层里面的第一个神经元；后面的数字跟特征编号是一样的——第一个神经元对第二个输入的参数就是 w_{12} 。所以我们来看一下对应的公式。

每个神经元都做同一套两步操作

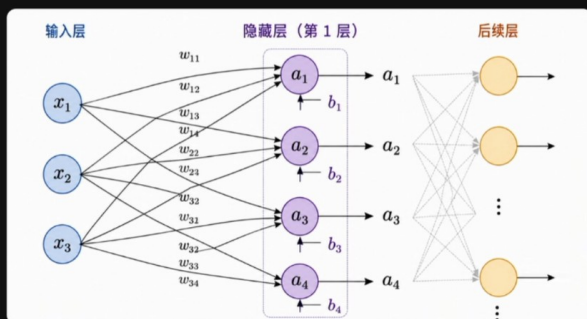
① 线性变换

$$z = w_1x_1 + w_2x_2 + \cdots + w_nx_n + b$$

② 非线性激活

$$a = \sigma(z)$$

差别只在于：每个神经元维护着自己的一套权重和偏置。以下图为例（3 输入 → 4 神经元）：



$$\begin{aligned} a_1 &= \sigma(w_{11}x_1 + w_{12}x_2 + w_{13}x_3 + b_1) \\ a_2 &= \sigma(w_{21}x_1 + w_{22}x_2 + w_{23}x_3 + b_2) \\ a_3 &= \sigma(w_{31}x_1 + w_{32}x_2 + w_{33}x_3 + b_3) \\ a_4 &= \sigma(w_{41}x_1 + w_{42}x_2 + w_{43}x_3 + b_4) \end{aligned}$$

图 10 | 每个神经元都做同一套两步操作（3 输入 → 4 神经元）

重点：每个神经元两步操作：① 线性变换（本质是线性回归）；② 非线性激活（ σ 是激活函数的通用符号，输出大称“激活”、否则“抑制”）。没有非线性，叠多少层都等价于一层线性变换。

$$z = w_1x_1 + w_2x_2 + w_3x_3 + b \quad a = \sigma(z)$$

权重下标： w_{ij} 中 i = 本层第 i 个神经元， j = 对上一层第 j 个输入。例如 w_{12} = 第 1 个神经元对第 2 个输入的权重。

如果写成通项：第 L 层中的第 i 个神经元，算出来的激活值是多少？就是激活函数包裹着求和： j 代表上一层的输入，从 1 到 n 一共有 n 个输入； w_{ij} 是第 i 个神经元的第 j 个输入（第 L 层），再乘上它的特征。把求和公式去掉，它就是向量的意思了。 x 其实就是上一层的输入：当前层是 L ，上一层就是 $L-1$ ，再加上 b_i 。这个公式就是一个通项——第 L 层第 i 个神经元做的运算就是这个。因为每个神经元都在做一样的运算，我们不会每一个单独计算；为了运算效率，会把同一层所有的神经元打包成一个矩阵，一次乘法就搞定。用矩阵表达时，小写一般是标量或向量，大写一般代表矩阵：参数是一个矩阵，再乘以上一层的输入（上一层的输出就是本层的输入），再加上 b （这也是一个向量），一次就能运算出来。这也是为什么神经网络都要用 GPU——GPU 就特别擅长并行做这种矩阵的运算。

从单神经元到整层矩阵化

第 l 层第 i 个神经元的输出（通项形式）：

$$a_i^{(l)} = \sigma \left(\sum_{j=1}^n w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)} \right)$$

为了运算效率，把同一层所有神经元打包成矩阵，一次乘法搞定整层：

$$\mathbf{A}^{(l)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{A}^{(l-1)} + \mathbf{b}^{(l)} \right)$$

这就是为什么神经网络计算天然适合 GPU——它就是一连串大型矩阵乘法。

图 11 | 从单神经元到整层矩阵化

重点：单神经元通项与整层矩阵形式——

$$a_i^{(l)} = \sigma \left(\sum_j w_{ij}^{(l)} a_j^{(l-1)} + b_i^{(l)} \right)$$

$$\mathbf{A}^{(l)} = \sigma \left(\mathbf{W}^{(l)} \mathbf{A}^{(l-1)} + \mathbf{b}^{(l)} \right)$$

上标 (l) 是层号， l 是当前层、 $l-1$ 是上一层；小写 = 标量/向量，大写 = 矩阵。矩阵化是为了运算效率，也是神经网络要用 GPU 的原因。

我们来看一下上面的例子：上一层的三个输入、这一层有四个神经元的时候会怎么写？每一层的神经元横着写，每个神经元有三个权重、一个偏置，四个神经元就是四个偏置。一个 4×3 的矩阵乘上 3×1 的向量（这就是三个输入），加起来，一次就把这四个神经元的值都算出来了，效率就比较高。这个就是前向传播的过程。

矩阵化示例：3 输入 $\rightarrow$ 4 神经元

$$\mathbf{A}^{(1)} = \sigma \left(\begin{bmatrix} w_{11} & w_{21} & w_{31} \\ w_{12} & w_{22} & w_{32} \\ w_{13} & w_{23} & w_{33} \\ w_{14} & w_{24} & w_{34} \end{bmatrix} \begin{bmatrix} a_1^{(0)} \\ a_2^{(0)} \\ a_3^{(0)} \end{bmatrix} + \begin{bmatrix} b_1^{(1)} \\ b_2^{(1)} \\ b_3^{(1)} \\ b_4^{(1)} \end{bmatrix} \right)$$

一次矩阵乘法就算完了 4 个神经元，比逐个循环高效几个数量级。

图 12 | 矩阵化示例：3 输入 $\rightarrow$ 4 神经元，一次乘法算完整层

重点：形状： $[4 \times 3] \cdot [3 \times 1] + [4 \times 1] \rightarrow [4 \times 1]$ ，一次矩阵乘法算完 4 个神经元，比逐个循环高效几个数量级。

六、损失函数与梯度下降

刚才前向传播算完了之后，要干什么？就要算损失有多大了。假如是一个回归问题，我们一般就会用均方差来解决：真实值减预测值的平方进行求和，再除以 N （如果想除以样本量，会有一些微调的操作，比如 $N-1$ 之类的，这个不影响整体的计算）。这就是损失函数的计算方法：损失值越大，就说明错得越离谱。那我们要做的，就是如何调整刚才的权重和偏置这些参数（权重矩阵 W 和偏置向量 b ），让这个 L 变小。在线性回归和逻辑回归中，其实我们已经接触过这种梯度下降：算出损失对每个参数的偏导，组成的向量就是梯度。这个梯度代表的含义，是损失函数增长最快的方向；想让损失下降，就沿着增长的负方向去调整一小点。比如对权重 W ：让它往负方向走，减去当前的梯度，再乘上一个小的 η ——学习率，每次不要一次调太大，调一个很小的数字，再重复进行运算。这个就是梯度下降的过程了。

谦行Aling bilibili

损失函数：把"错得多离谱"量化

前向传播得到 $\hat{y}$ ，但它和真实标签 y 总有差距。我们用**损失函数**把这个差距量化为一个数：

线性回归常用 MSE（均方误差）：

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2$$

$\mathcal{L}$ 越大，说明错得越离谱。问题随之而来：该怎么调整权重矩阵 W 和偏置 b ，才能让 $\mathcal{L}$ 变小？

线性回归、逻辑回归里的答案是**梯度下降**：算出损失对每个参数的偏导，组成向量——**梯度**，它指向损失**上升最快**的方向。要让损失下降，沿**负梯度**方向走一小步即可：

$$W \leftarrow W - \eta \cdot \frac{\partial \mathcal{L}}{\partial W}$$

图 13 | 损失函数与梯度下降

重点： 回归问题最常用的损失函数是 MSE（均方误差）——

$$L = (1/N) \sum_i (y_i - \hat{y}_i)^2$$

N 为样本数， L 越大 = 错得越离谱。**梯度 = 损失上升最快的方向**，想让损失下降就沿负梯度方向走一小步：

$$W \leftarrow W - \eta \cdot \partial L / \partial W$$

η （读“伊塔”）是学习率：控制每次只调一小点，不要一次调太大；调整后重复运算，循环往复。

七、链式法则：反向传播的数学核心

有了这个我们就可以调梯度了。但是在神经网络中，实际上要复杂非常多。为什么呢？因为每层的输出刚才我们已经看到了，它又是下一层的输入：第 $L-1$ 层的输出，在 L 层又是输入——这就是典型的复合函数，是多层函数进行嵌套；而且神经网络非常多层，它就嵌套了很多层。那么要怎么计算一个复合函数的导数呢？这就要利用数学中讲的链式法则了。

问题：神经网络计算梯度

谦行Aling bilibili

神经网络会复杂很多，每层输出又是下一层的输入：

$$a^{(l)} = \sigma \left(\sum_{i=1}^n w_i a^{(l-1)} + b \right)$$

这是典型的复合函数嵌套，而且嵌套了好多层。要算损失对深处某个参数的偏导，必须借助强大的数学工具——链式法则。

图 14 | 每层输出是下层输入：典型的复合函数

重点：神经网络 = 多层嵌套的复合函数（每层输出是下层输入）→ 求导必须用链式法则。

带着大家来看一些数学知识，这也是理解反向传播最重要的点。如果变量 x 能影响一个变量 u ，而 u 又会影响 y ，那么根据链式法则， x 对 y 的总影响是多少？就是 x 对 u 的影响，乘上 u 对 y 的影响——两段影响的乘积。用数学公式表达： $y = f(u)$ （ u 影响 y ）， $u = g(x)$ （ x 影响 u ），那么 x 对 y 的影响，就是 y 对 x 的导数 = y 对 u 的导数乘上 u 对 x 的导数。这就是链式法则最基本的规定。

链式法则的朴素思想

谦行Aling bilibili

如果 x 影响 u ， u 又影响 y ，那么 x 对 y 的总影响 = 两段影响的乘积。

若 $y = f(u)$ ， $u = g(x)$ ，则：

$$\frac{dy}{dx} = \frac{dy}{du} \cdot \frac{du}{dx}$$

重点：链式法则——

$$dy/dx = dy/du \cdot du/dx$$

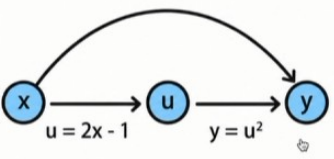
$x \rightarrow u \rightarrow y$ 的总影响 = 两段影响的乘积 ($y = f(u)$, $u = g(x)$)。

带着大家看个例子： $y = u^2$, $u = 2x - 1$ 。我们可以画一个图形来表达这个关系，这个图形就称之为计算图。 y 对 x 的导数：先乘 y 对 u 的导数（幂函数求导，是 $2u$ ），再乘上 u 对 x 的导数（常数 2），就是 $2u \times 2 = 4u$ 。算出 $4u$ 之后，因为要算的是 y 对 x 的导数，最终还得用 x 来表达，把 $u = 2x - 1$ 代入，就等于 $8x - 4$ 。导数的含义：表示当 x 取某个值的时候，比如 $x = 1$ ，我们算出 $8 \times 1 - 4 = 4$ ，那么在 $x = 1$ 附近， x 变化一小点， y 会相应地变化多少——比如说 x 变化 0.001， y 就会相应地变化 0.004。这个就是导数的含义。

谦行Aling bilibili

示例：用计算图理解链式法则

设 $y = u^2$, $u = 2x - 1$ ，画成计算图：



$$\begin{aligned} \frac{du}{dx} &= 2 & \frac{dy}{du} &= 2u \\ \frac{dy}{dx} &= \frac{dy}{du} \cdot \frac{du}{dx} \\ &= (2u) \cdot 2 \\ &= 4u \\ &= 4(2x - 1) \\ &= 8x - 4 \end{aligned}$$

$$\begin{aligned} \frac{dy}{dx} &= \frac{dy}{du} \cdot \frac{du}{dx} \\ &= 2u \cdot 2 \\ &= 4(2x - 1) \\ &= 8x - 4 \end{aligned}$$

代入 $x = 1$: $\frac{dy}{dx} = 4$ 。
即 x 每增加 0.001, y 大约增加 0.004。

图 16 | 计算图示例： $y = u^2$, $u = 2x - 1$

重点：计算图 = 画出变量依赖关系的图。本例求导：

$$dy/dx = 2u \times 2 = 4u = 4(2x - 1) = 8x - 4$$

导数含义： $x = 1$ 时导数为 4，即 x 变化 0.001， y 约变化 0.004。

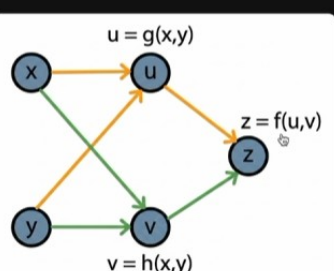
我们再来看一下比较复杂的情况：多变量——不只是受一个变量的影响，它受两个变量。看这个计算图： z 受 u 和 v 两个变量的影响，而 u 又是受 x 和 y 的影响， v 也是受 x 、 y 的影响。这种时候怎么算 z 对 x 的导数？其实也是这样： z 对 x 的影响，实际上是通过 u 这一条链条和 v 这一条链条过来的，其他的路径影响不了它。那就把这两条路径上的导数都算出来——每条路径上就变成对单变量求导，跟刚才讲的一样——再把两个结果相加： z 对 x 的导数 = z 对 u 的导数乘 u 对 x 的导数，加上 z 对 v

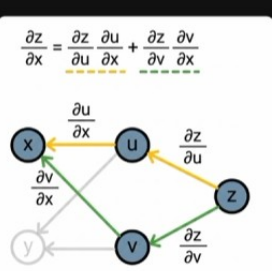
的导数乘 v 对 x 的导数。这就是复合函数求导，称之为偏导数：因为 z 还受 y 的影响，但只考虑 x 对它的影响时， y 就变成常数了，所以称之为偏导。

谦行Aling bilibili

多变量：所有路径导数之和

设 $z = f(u, v)$ ，而 $u = g(x, y)$ 、 $v = h(x, y)$ 。此时 x 沿 **两条路径** 影响 z ： $x \rightarrow u \rightarrow z$ 与 $x \rightarrow v \rightarrow z$ 。每条路径用单变量链式法则算出导数，**再把它们加起来**。



$$\frac{\partial z}{\partial x} = \frac{\partial z}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial z}{\partial v} \frac{\partial v}{\partial x}$$


$$\frac{\partial z}{\partial x} = \underbrace{\frac{\partial z}{\partial u} \cdot \frac{\partial u}{\partial x}}_{x \rightarrow u \rightarrow z} + \underbrace{\frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial x}}_{x \rightarrow v \rightarrow z}$$

图 17 | 多变量偏导：分路径求导，再相加

重点：多变量情况：每条路径连乘、各路径再相加——

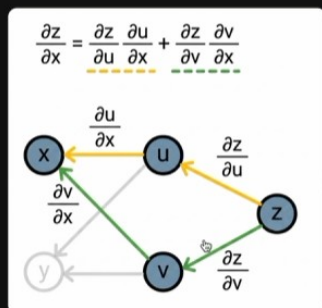
$$\partial z / \partial x = \partial z / \partial u \cdot \partial u / \partial x + \partial z / \partial v \cdot \partial v / \partial x$$

只考虑 x 的影响、把 y 当常数时求出的导数，就叫偏导数。

举个例子： $z = u^2 + v^2$ ， $u = x + y$ ， $v = x - y$ ，求 z 对 x 的导数。先算 z 对 u 的导数乘上 u 对 x 的导数： z 对 u 的导数是 $2u$ （这时 v 就是常量，不考虑了）； u 对 x 的导数， y 是常量， x 的导数就是 1。再加上 z 对 v 的导数乘 v 对 x 的导数： z 对 v 的导数是 $2v$ （这时 u 当作常量）； v 对 x 的导数也是 1。所以加起来就是 $2u + 2v$ ，再把 $u = x + y$ 、 $v = x - y$ 代入，一抵消就变成了 $4x$ 。这种时候就能算出来，在复合的情况下， x 的变化会对 z 的变化有多大的影响。这个其实就是反向传播，也就是接下来神经网络里依赖的最核心的公式。那反向传播是怎么沿着这个计算图一层层地来找“背锅侠”、每一个该背多少锅的呢？前向传播就是沿着计算图算最后的预测值；反向传播就是用链式法则一层层地求导、算出最后的导数来，其实我们在过程中也会把这些中间结果保存下来。

多变量小例题

$z = u^2 + v^2$, $u = x + y$, $v = x - y$, 求 $\frac{\partial z}{\partial x}$:



$$\begin{aligned}\frac{\partial z}{\partial x} &= \frac{\partial z}{\partial u} \cdot \frac{\partial u}{\partial x} + \frac{\partial z}{\partial v} \cdot \frac{\partial v}{\partial x} \\ &= 2u \cdot 1 + 2v \cdot 1 \\ &= 2(x + y) + 2(x - y) \\ &= 4x\end{aligned}$$

两条路径的影响分别相乘再相加——这正是反向传播在分叉节点处的核心操作。

只考虑x对它的影响的时候

图 18 | 计算图小结：前向算值，反向求导

重点：例算： $2u \times 1 + 2v \times 1 = 2u + 2v = 2(x+y) + 2(x-y) = 4x$ 。

计算图上的双向流动： 前向沿图从左到右算预测值；反向沿图从右到左逐层求导；过程中的中间结果会保存下来，反向传播时直接复用。

能用 PyTorch 的时候，我们就不用自己手工做刚才的计算了——PyTorch 会自动微分，把反向传播的导数都算出来。

反向传播——沿计算图寻找"背锅侠"

→ 前向传播

沿箭头方向计算每个节点的值

← 反向传播

逆着箭头方向，用链式法则把局部梯度一路乘回输入端

计算图的妙处：复杂的多元复合求导被简化成"沿路径相乘局部梯度"。每个节点只需关心自己的输入输出——这正是 PyTorch 等框架自动微分的基石。

图 19 | PyTorch 自动微分

重点： PyTorch 的 autograd 会自动构建计算图、自动完成全部反向求导——但先手推一遍最简单的例子，把原理吃透，剩下的交给它代劳。

八、手推反向传播：极简网络完整实例

我们先不用 PyTorch，自己手工推演一个：如果能把一个最简单的网络推演出来，原理就理解了；理解了原理之后，用 PyTorch 帮我们代劳就行了。先来构建一个极简的神经网络：三层，每层就一个神经元；为了让大家看清楚计算过程，都不算偏置了（偏置都是零）。输入是 1，我们想预测的值是 0。一开始为每个神经元初始化权重：分别是 2、3、-1。激活函数用 ReLU：当 z 小于零的时候，输出永远是零；当 z 大于零的时候，输出就是 z 自己——这在计算导数的时候非常方便。损失函数，因为这是一个回归问题，用 MSE 就行了。

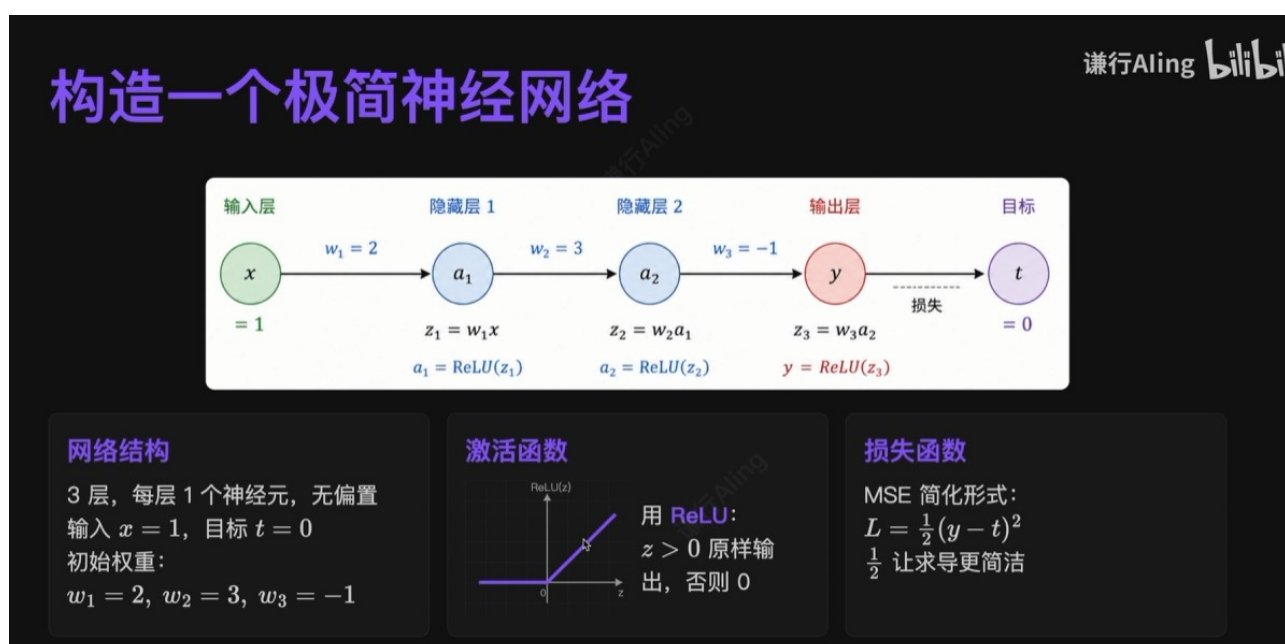


图 20 | 构造极简神经网络：结构、激活函数、损失函数

重点：网络设定：3 层 × 每层 1 个神经元，无偏置；输入 $x = 1$ ，目标 $t = 0$ ；初始权重 $w_1 = 2$ 、 $w_2 = 3$ 、 $w_3 = -1$ ；激活 ReLU；损失用 MSE 的简化形式（ $\frac{1}{2}$ 让求导更干净）：

$$L = \frac{1}{2}(y - t)^2$$

先看一下前向传播。前向传播中，要把每个节点算出来的值记下来——反向传播的时候还要用，最终来算出预测值是多少。输入是 $x = 1$ ：第一层 $z_1 = w_1 x$ ， w_1 初始化的权重是 2，就是 $2 \times 1 = 2$ ；经过激活函数， $\text{ReLU}(2)$ ：只要小于零的时候都等于零，大于零的时候就等于自己，所以 $\text{ReLU}(2) = 2$ 。

“先呢看一下前向传播，在全向前向传播中，我们要把每个节点算出来的值呢给它记下来，在反向传播的时候，我们还要用最终来算出预测值是多少。来看一下我们的输入是 $x=1$ ，第一层 $z = w_1 * x$ ， w_1 我们初始化的权重是 2，就是 $2 \times 1 = 2$ ；经过激活函数，我们会发现 $\text{ReLU}(z_1)$ 呢，只要你小于零的时候就都等于零，你大于零的时候呢就等于自己，所以 $\text{ReLU}(2) = 2$ ”

图 21 | 前向传播讲解（视频字幕原图）

把 z_1 和 a_1 记下来，然后再往下传播： $z_2 = w_2 a_1$ （上一层的输出就是下一层的输入）， w_2 初始化为 3，就是 $3 \times 2 = 6$ 。同样地一层层就能算出 z_3 ： w_3 是 -1 ， $z_3 = (-1) \times 6 = -6$ ； $y = z_3$ ，不做变换，预测值就是 -6 。我们想预测的是 0，来算一下整个的损失有多大： $L = \frac{1}{2}(-6 - 0)^2 = 18$ 。这个还是比较大的——预测值是 -6 ，真实值是 0，离目标有点远，损失高达 18。那么我们就利用反向传播，看看这个 18 的损失，由这一层层的几个参数分别该背多少锅。

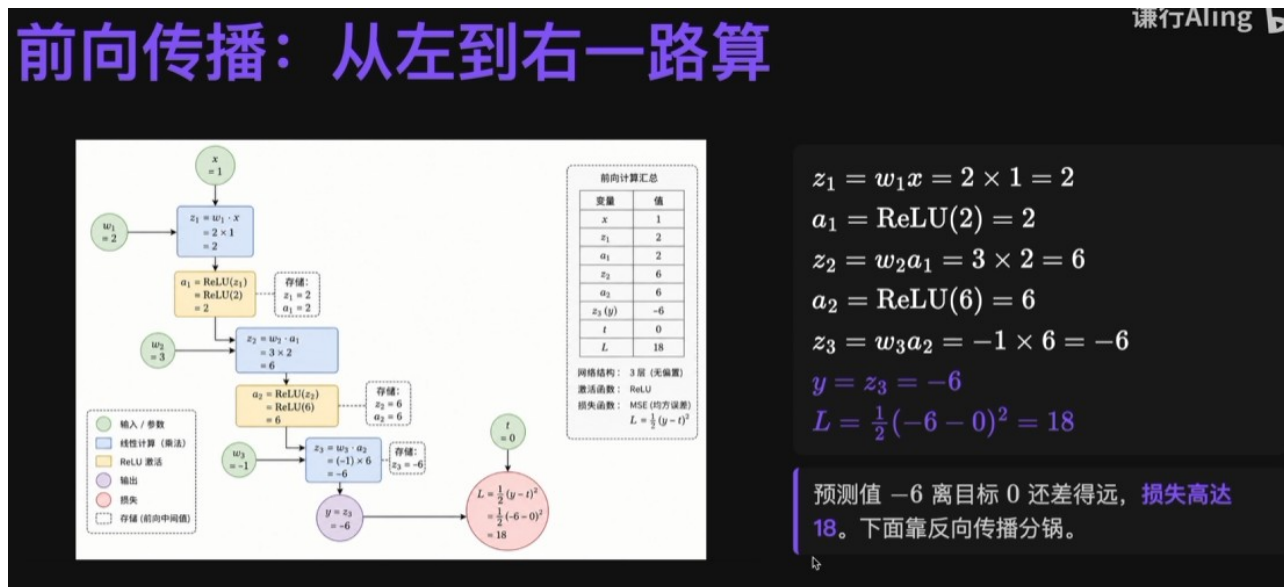


图 22 | 前向传播：从左到右一路算，中间值存起来备用

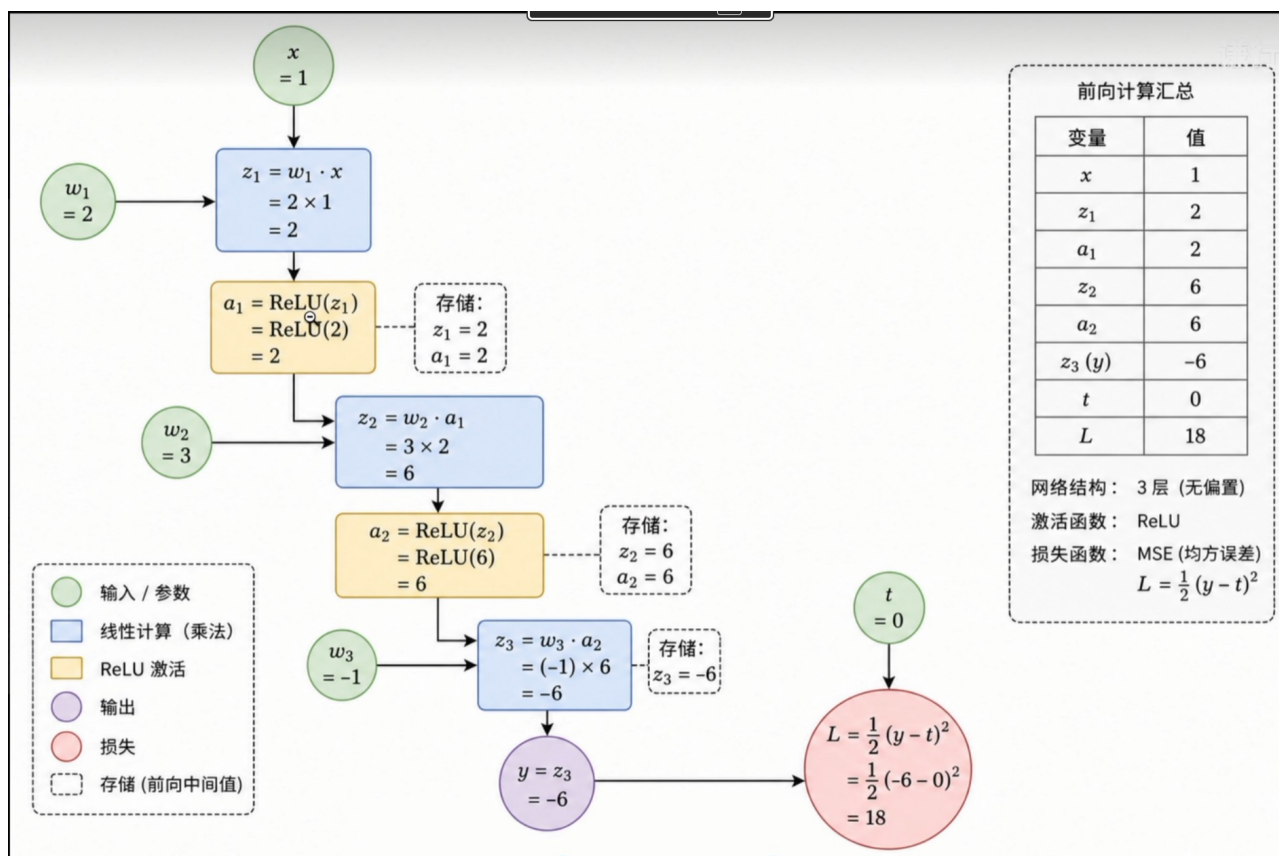


图 23 | 前向传播计算图与数值汇总

重点：前向传播数值汇总：

变量	数值	含义
x	1	输入
z_1	2	第一层线性结果 ($w_1x = 2 \times 1$)
a_1	2	$\text{ReLU}(2) = 2$
z_2	6	第二层线性结果 ($w_2a_1 = 3 \times 2$)
a_2	6	$\text{ReLU}(6) = 6$
$z_3 (= y)$	-6	输出层线性结果 ($w_3a_2 = -1 \times 6$)，即预测值
t	0	目标值
L	18	损失 $\frac{1}{2}(-6 - 0)^2 = 18$

在开始算之前，先给大家介绍一下 ReLU 的导数。之所以用 ReLU，也是因为它导数特别简单：当大于零的时候，输出永远是 z ，那导数就是 1；小于零的时候，输出永远是零，导数就是 0。当然在等于零的时候，单纯这一个点严格来说是不可导的，记成 1 也行、记成 0 也行；实际的神经网络中很少有正好等于零的情况，所以说就没关系了。

谦行Aling bilibili

先准备好 ReLU 的导数

$$\text{ReLU}'(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$$

ReLU 比阶跃函数好在哪？

- 处处可导（0 处规定为 1），可参与反向传播
- 导数极其简单，大幅提升运算速度

图 24 | 先准备好 ReLU 的导数

重点：ReLU 的导数——

$$\text{ReLU}'(z) = 1 \quad (z > 0) ; \quad = 0 \quad (z \leq 0)$$

$z = 0$ 处严格不可导，工程上规定成 1 或 0 都行，实际几乎不影响。ReLU 比阶跃函数好：处处可导（能参与反向传播）+ 导数极简（运算快）。

我们来看看从损失 18 开始，利用链式法则层层往回推导，看看每个参数应该对最终的误差承担多大的责任。这个责任把它用向量连起来，就是一个梯度。先从最终的损失函数开始：损失函数由 y 计算

来，求它对 y 的导数，把 2 底乘过来就是 $y - t = -6 - 0 = -6$ 。接下来算 L 对 z_3 的导数： z_3 是通过 $y = z_3$ 来的，先乘 L 对 y 的导数（ -6 ——上一层刚算出来，这也是反向传播的意义：每一步用的都是上一次运算的结果），再乘 y 对 z_3 的导数（ $y = z_3$ ，所以是 1），还是 -6 。再往回就到了 w_3 ——这其实就是我们想算的第一个值： L 对 w_3 的偏导 = L 对 z_3 的导数（ -6 ）乘上 z_3 对 w_3 的导数（其实就是 a_2 ，把它当常数，而 a_2 在前向传播时已经算出来了，是 6），代入进来就是 $-6 \times 6 = -36$ 。可见 w_3 只要稍微动一点，影响就是非常大的。

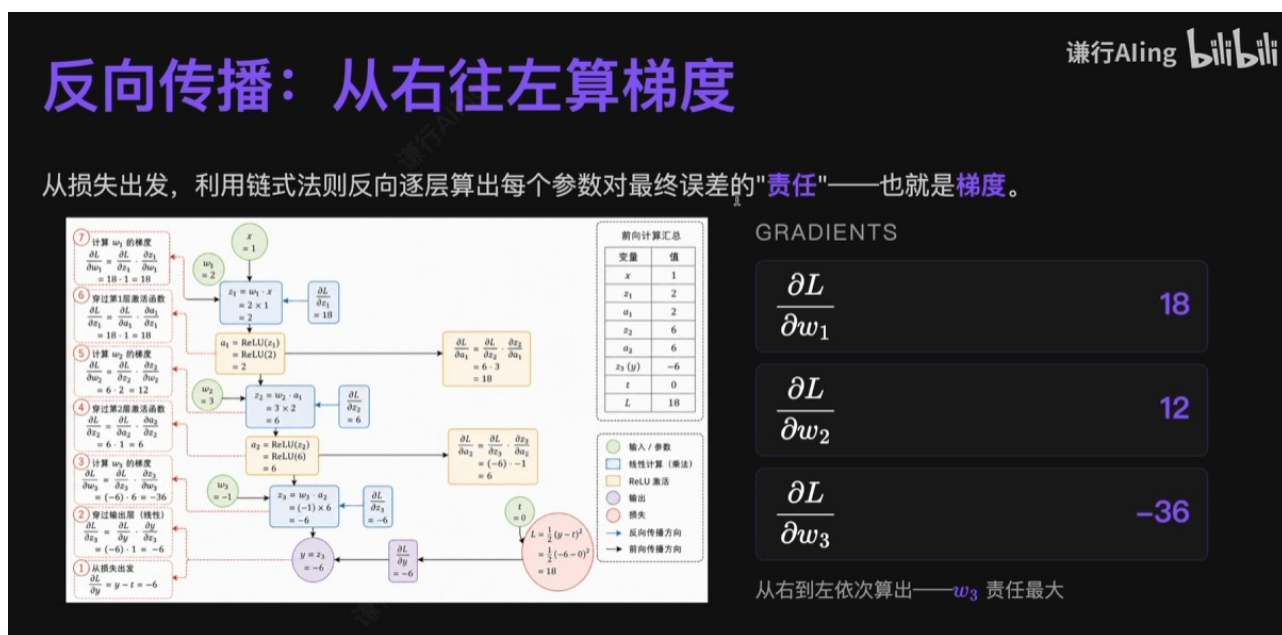


图 25 | 反向传播：从右往左算梯度

再往回传一层，就到了上一层的非线性激活层（ReLU）。先算 L 对 a_2 的偏导： a_2 是通过 z_3 影响 L 的，就是 L 对 z_3 的导数（ -6 ）乘上 z_3 对 a_2 的导数（就是 $w_3 = -1$ ）， $-6 \times (-1) = 6$ 。有了 L 对 a_2 的导数，再看 ReLU 这一层的导数： L 对 z_2 的导数要经过 a_2 来算， L 对 a_2 的导数（6）乘上 a_2 对 z_2 的导数（就是 ReLU 的导数—— z_2 明显大于零，是 6，所以导数是 1），算出来还是 6。再往上一层，终于可以算对 w_2 的导数了： w_2 是通过 z_2 影响损失函数的， L 对 w_2 的导数 = L 对 z_2 的导数（6，上一层刚算出来）乘上 z_2 对 w_2 的导数（其实就是 a_1 ，前向传播时算出来了，是 2）， $6 \times 2 = 12$ 。对 w_1 的算法也是一样的，只要耐心地逐层推导，就能算出整个过程。

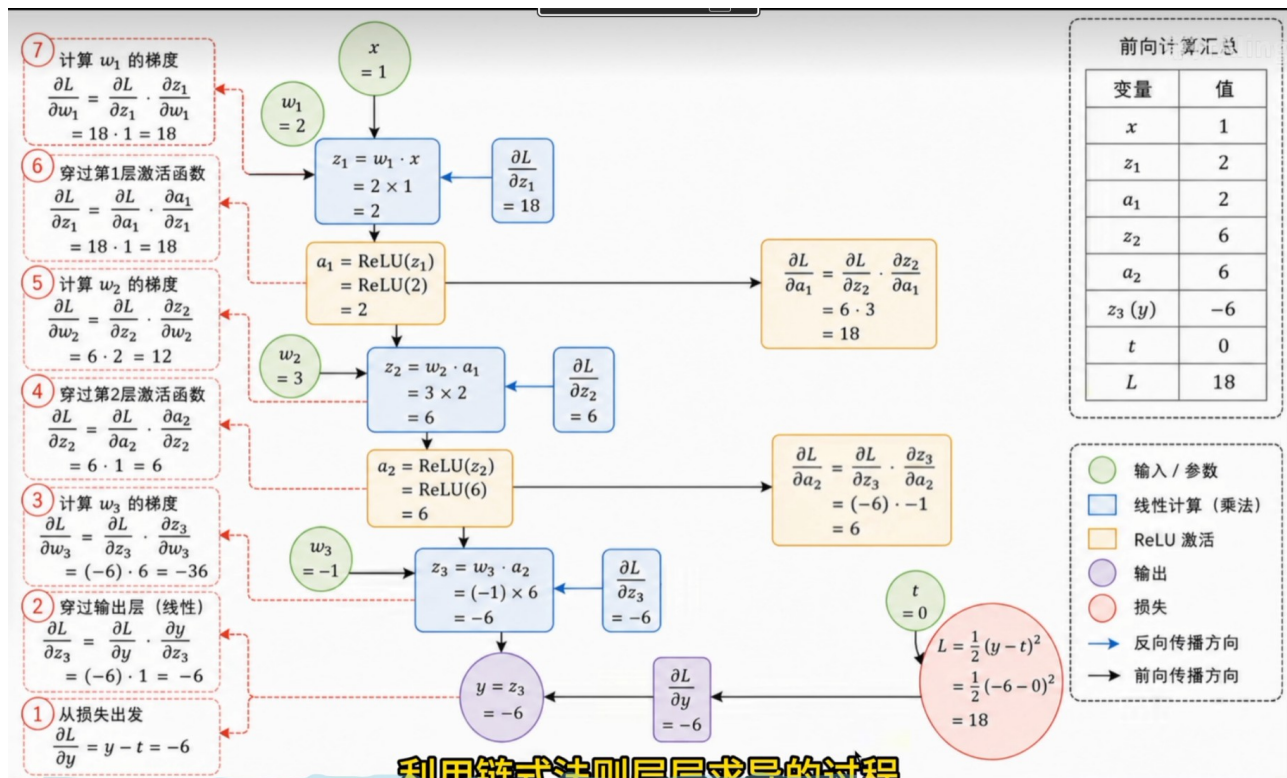


图 26 | 反向传播计算图：梯度沿虚线逐步回传

最终就能算出来对 w_1 、 w_2 、 w_3 的偏导。大家也能看出来， w_3 的影响是最大的，剩下两个的影响也不小。

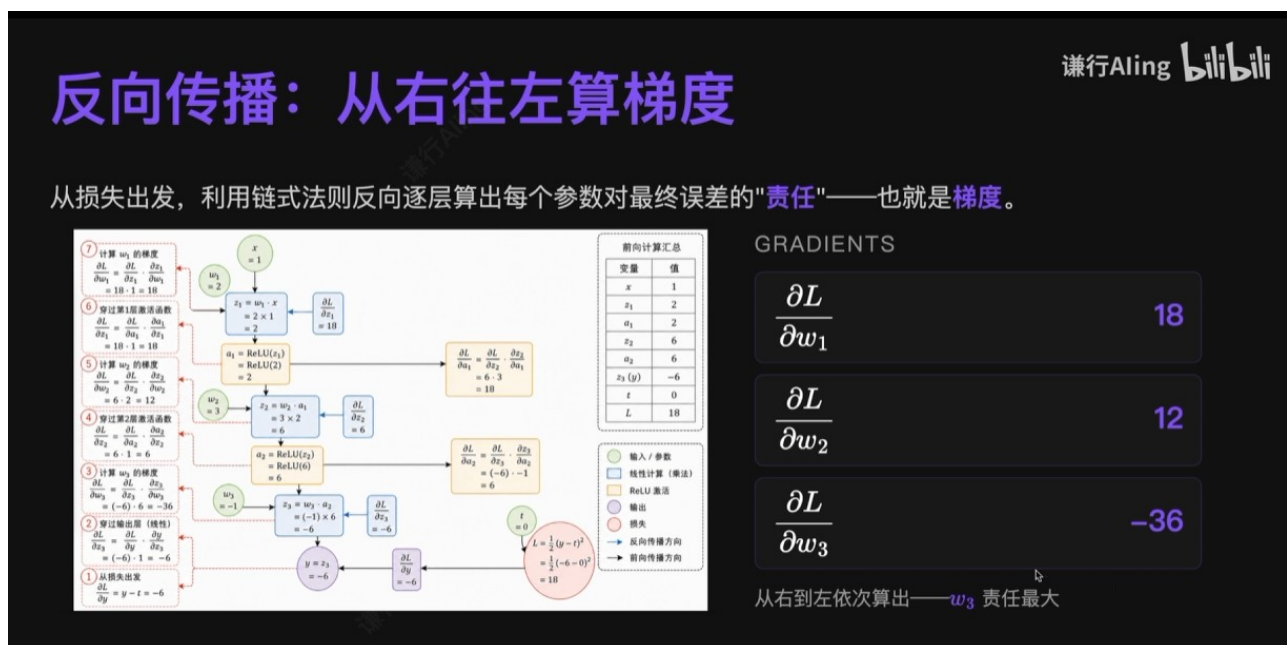


图 27 | 梯度结果： w_3 的责任最大

重点：完整梯度链条共 7 步，每步都复用上一步的结果：

步骤	求什么	计算过程	结果
①	$\partial L / \partial y$	$y - t = -6 - 0$	-6

②	$\partial L / \partial z_3$	$\partial L / \partial y \cdot 1$ ($y = z_3$, 输出层无激活)	-6
③	$\partial L / \partial w_3$	$\partial L / \partial z_3 \cdot a_2 = (-6) \times 6$	-36
④	$\partial L / \partial a_2, \partial L / \partial z_2$	$\partial L / \partial z_3 \cdot w_3 = (-6) \times (-1) = 6$; 再 $\times \text{ReLU}'(z_2) = 1$ ($z_2 = 6 > 0$)	6、6
⑤	$\partial L / \partial w_2$	$\partial L / \partial z_2 \cdot a_1 = 6 \times 2$	12
⑥	$\partial L / \partial a_1, \partial L / \partial z_1$	$\partial L / \partial z_2 \cdot w_2 = 6 \times 3 = 18$; 再 $\times \text{ReLU}'(z_1) = 1$ ($z_1 = 2 > 0$)	18、18
⑦	$\partial L / \partial w_1$	$\partial L / \partial z_1 \cdot x = 18 \times 1$	18

详细过程如下：

详细计算 ①：从损失到 w_3

第一步：从损失出发

$$\frac{\partial L}{\partial y} = (y - t) = -6$$

第二步：穿过输出层 ($y = z_3$)

$$\frac{\partial L}{\partial z_3} = \frac{\partial L}{\partial y} \cdot 1 = -6$$

第三步：算 w_3 的梯度 ($z_3 = w_3 a_2$)

$$\frac{\partial L}{\partial w_3} = \frac{\partial L}{\partial z_3} \cdot a_2 = -6 \times 6 = -36$$

图 28 | 详细计算 ①：从损失到 w_3 (第 1~3 步)

详细计算 ②：穿过 ReLU，算 w_2

第四步：传回 a_2 ，再穿过 ReLU 到 z_2

$$\begin{aligned} \frac{\partial L}{\partial a_2} &= \frac{\partial L}{\partial z_3} \cdot w_3 = -6 \times (-1) = 6 \\ \frac{\partial L}{\partial z_2} &= \frac{\partial L}{\partial a_2} \cdot \text{ReLU}'(z_2) = 6 \times 1 = 6 \end{aligned}$$

这里就能看出保留前向中间值的意义：要靠 $z_2 = 6 > 0$ 才能判断 ReLU 导数为 1。

第五步：算 w_2 的梯度 ($z_2 = w_2 a_1$)

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial z_2} \cdot a_1 = 6 \times 2 = 12$$

图 29 | 详细计算 ②：穿过 ReLU 算 w_2 (第 4~5 步)

详细计算 ③：到底层 w_1

谦行Aling bilibili

第六步：传回 a_1 ，再穿过 ReLU 到 z_1

$$\frac{\partial L}{\partial a_1} = \frac{\partial L}{\partial z_2} \cdot w_2 = 6 \times 3 = 18$$

$$\frac{\partial L}{\partial z_1} = 18 \times \text{ReLU}'(z_1) = 18 \times 1 = 18$$

第七步：算 w_1 的梯度 ($z_1 = w_1 x$)

$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial z_1} \cdot x = 18 \times 1 = 18$$

至此，三个权重的梯度全部算完。

图 30 | 详细计算 ③：到底层 w_1 (第 6~7 步)

重点：三个关键观察：① 每步只用两样东西——上一步刚算出的导数 + 前向传播保存的中间值 (a_1 、 a_2 的数值、 z_1 、 z_2 的正负号)，这就是前向要存中间值的意义：要靠 $z_2 = 6 > 0$ 才能判断 ReLU 导数为 1；② $|\partial L / \partial w_3| = 36$ 最大—— w_3 责任最大、稍微动一点对损失影响就很大；③ 梯度为负（如 -36 ）意味着该参数增大反而会让损失下降。

梯度结果与符号含义

谦行Aling bilibili

参数梯度	梯度值	含义
$\frac{\partial L}{\partial w_1}$	18	w_1 增加 1，损失增加约 18
$\frac{\partial L}{\partial w_2}$	12	w_2 增加 1，损失增加约 12
$\frac{\partial L}{\partial w_3}$	-36	w_3 增加 1，损失减少约 36

梯度正 → 该参数增大会让损失上升，要减小它；梯度负 → 反之。这就是 $-\eta \cdot \nabla$ 的来历。

图 31 | 梯度结果与符号含义

重点：梯度算完后的符号含义：

梯度	含义
$\partial L / \partial w_1 = 18$	w_1 增大 1，损失约上升 18 → 应减小 w_1

$\partial L / \partial w_2 = 12$	w_2 增大 1, 损失约上升 12 → 应减小 w_2
$\partial L / \partial w_3 = -36$	w_3 增大 1, 损失约下降 36 → 应增大 w_3

梯度为正 → 该参数增大会让损失上升, 要减小它; 梯度为负 → 反之。这就是更新公式里 “ $-\eta \cdot$ 梯度” 的来历。

有了梯度, 就可以用梯度下降更新参数了。设学习率 $\eta = 0.01$, 按照更新公式 $w \leftarrow w - \eta \cdot \partial L / \partial w$, 对三个权重逐个更新。注意 w_3 : 它的梯度为负, 减去一个负数等于加上 0.36, 所以 w_3 从 -1 调整为 -0.64 , 朝增大的方向走。

谦行Aling bilibili

用梯度下降更新参数

设学习率 $\eta = 0.01$, 公式:

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$

参数	旧值	梯度	更新量 $-\eta \cdot \nabla$	新值
w_1	2	18	$-0.01 \times 18 = -0.18$	1.82
w_2	3	12	$-0.01 \times 12 = -0.12$	2.88
w_3	-1	-36	$-0.01 \times (-36) = +0.36$	-0.64

图 32 | 用梯度下降更新参数 ($\eta = 0.01$)

重点: 参数更新 ($\eta = 0.01$) :

参数	旧值	梯度	更新量 $-\eta \cdot \partial L / \partial w$	新值
w_1	2	18	$-0.01 \times 18 = -0.18$	1.82
w_2	3	12	$-0.01 \times 12 = -0.12$	2.88
w_3	-1	-36	$-0.01 \times (-36) = +0.36$	-0.64

更新完参数, 再跑一次前向传播来验证效果: 用新的权重重新算一遍。

验证：再跑一次前向传播

$$\begin{aligned} z_1 &= 1.82 \times 1 = 1.82, & a_1 &= 1.82 \\ z_2 &= 2.88 \times 1.82 \approx 5.2416, & a_2 &\approx 5.2416 \\ z_3 &= (-0.64) \times 5.2416 \approx -3.3546 \\ y' &\approx -3.3546 \\ L' &= \frac{1}{2}(-3.3546)^2 \approx 5.63 \end{aligned}$$

更新前
L = 18
预测值 -6

更新后
L ≈ 5.63
预测值 -3.35

图 33 | 验证：再跑一次前向传播，损失明显下降

重点：验证结果（一轮迭代）：

指标	更新前	更新后（一轮迭代）
预测值 y	-6	≈ -3.35
损失 L	18	≈ 5.63

一轮迭代后损失从 18 降到约 5.63，预测值从 -6 拉近到约 -3.35——梯度下降确实在起作用。继续循环（再前向、再反向、再更新），损失会越来越接近 0。

九、总结：神经网络学习的本质

神经网络学习的本质，就是这样一个循环。

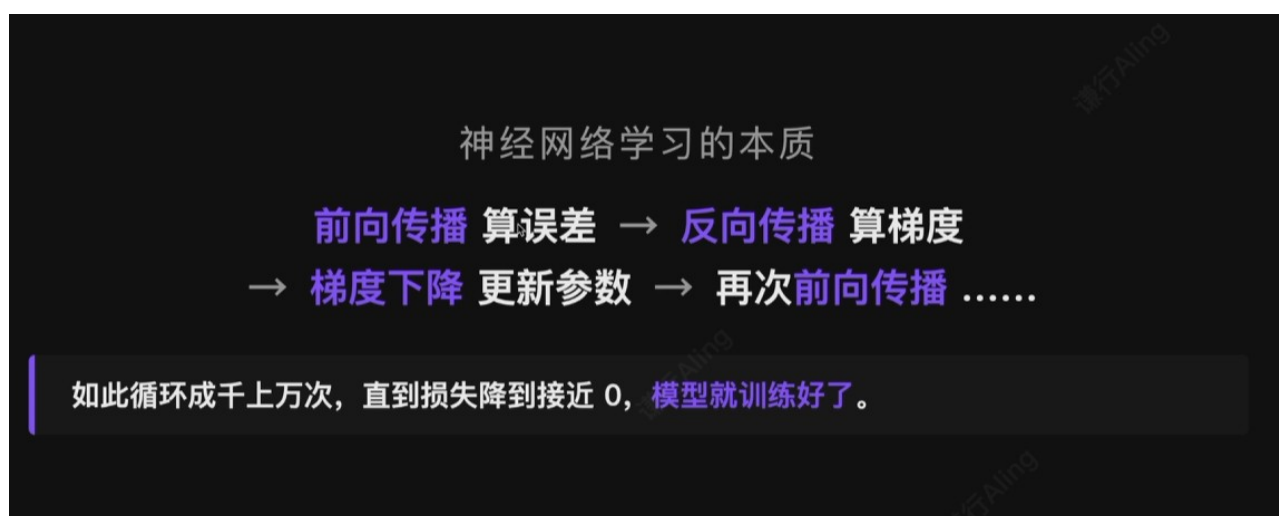


图 34 | 神经网络学习的本质

重点：一句话总结——

前向传播算误差 → 反向传播算梯度 → 梯度下降更新参数 → 再次前向传播 →

如此循环成千上万次，直到损失降到接近 0，模型就训练好了。

用追责类比串起来：反向传播 = 用链式法则沿计算图反向“分锅”（定责任）；梯度 = 责任大小的量化（正负号代表往哪边调）；学习率 η = 罚款的力度系数，每次只罚一小步；训练 = 分锅 → 罚款 → 再分锅 → 再罚款.....直到几乎没人该背锅（损失趋近 0）。

十、附录

附录 A：术语与勘误对照表

原笔记是语音转写式记录，部分名词被听错了，复习时按下表纠正：

原笔记写法	正确写法	说明
reload / re 录 / RELUT	ReLU	修正线性单元，本例使用的激活函数
偏执	偏置 (bias)	每个神经元自带的可学习参数
节约函数	阶跃函数	早期 MLP 使用的激活函数
西格玛	σ	激活函数的通用数学符号
辛顿	Hinton	Geoffrey Hinton, 1986 年推动反向传播与神经网络结合
均方差	均方误差 (MSE)	回归问题常用损失函数
学习器	优化器 (optimizer)	在梯度下降基础上改进的更新算法，如 SGD、Adam
丞相 (口误)	乘上	语音转写错误
“算出六来”	$y = -6$	本例预测值是负六： $w_3 = -1$, $z_3 = -1 \times 6 = -6$

附录 B：PyTorch 实现代码（用到再看）

实际用 PyTorch 时，它会自动构建计算图，调用 `loss.backward()` 就能自动完成全部反向传播求导，不需要手工推。图中代码用 `nn.Linear(1, 1, bias=False) + nn.ReLU()` 搭出的，正是与本笔记手推例子一致的结构。

用 PyTorch 实现：自动微分的优雅

只需写出前向公式，PyTorch 会在后台自动构建计算图，再用一行 `loss.backward()` 把所有梯度一次算出来。

```
import torch
import torch.nn as nn

# — 数据 —
x = torch.tensor([[1.0]])
t = torch.tensor([[0.0]])

# — 网络 —
model = nn.Sequential(
    nn.Linear(1, 1, bias=False), # w1
    nn.ReLU(),
    nn.Linear(1, 1, bias=False), # w2
    nn.ReLU(),
```

图 35 | PyTorch 实现：自动微分的优雅

附录 C：未完待续

原笔记标注“未完待续”。其中「激活函数的选择」「分类损失的推导」这两块，前面的手稿已经推进了一部分（ReLU / 池化 / 卷积的梯度、softmax + 交叉熵）；还剩以下主题，下次继续补充：

- 激活函数的选择；
- 各种优化器的作用；
- 梯度消失与梯度爆炸。